

Analysis: Aerobics

The key to this problem lies in realizing that there really is a lot of space on the mat to take advantage of, and a number of different approaches will work.

For the small input, putting the circles along one of the longer edge, and - if it runs out - along the opposite edge will work. The precise analysis of why they fit is somewhat tedious, so we will skip it in favor of describing two solutions that can also deal with the large input.

A randomized solution

The large input is a more interesting case. We will describe two solutions that allow one to solve this problem. The first one will be randomized. (If you solved the Equal Sums problem in Round 1B, you should already realize how helpful randomized algorithms can be!).

We will order the circles by decreasing radius. We will then take the circles one by one, and for each circle try to place it on the mat at a random point (that is, put the center of the circle at a random point on the mat). We then check whether it collides with any circle we have placed previously (by directly checking all of them). If it does collide, we try a different random point and repeat until we find one that works. When we succeed in placing all the circles, we're done.

Of course, if we manage to place all the circles, we have found a correct solution. The tricky part is why we will always find a good place to put the new circle in a reasonable time. To see this, let's consider the set of "bad points" - the points where we cannot put the center of a new circle because it would cause a collision. If we are now placing circle j with radius r_j , then it will collide with a previously placed circle i if and only if the distance from the new center to the center of circle i is less than $r_i + r_j$. This means that the "bad points" are simply a set of circles with radii $r_i + r_j$.

What is the total area of the set of bad points? Well, since the set is a group of circles, the area is at most the sum of the areas of the circles. (It can be less because the circles can overlap, but it cannot be more). As we are placing the circles ordered by decreasing radius, we know $r_j \leq r_i$, so the area of the i^{th} "bad" circle is at most $\pi * (2r_i)^2 = 4\pi r_i^2$. Here is where we will use that the mat is so large - we see that the total bad area is always at most 80 percent of the mat. Therefore, we have at least a 1 in 5 chance of choosing a good center on every attempt. In particular, it is always possible to find a good center, and it will take us only a few tries.

For each attempt, we have to make $O(N)$ simple checks, and we expect to make at most $5N$ attempts, so the expected time complexity of this algorithm is $O(N^2)$ - easily fast enough.

A deterministic solution

As usual, if we are willing to deal with a bit more complexity in the code, we can get rid of randomness in the solution, taking advantage of all the extra space we have in a different way. One sample way to do this follows.

To each circle of radius R we attach a $4R * 2R$ rectangle as illustrated below:



The top edge of the rectangle passes through the center of the circle. The bottom edge, the left edge, and the right edge are all at a distance of $2R$ from the center of the circle.

We now place circles one at a time, starting from the ones with the larger radius. We always place each circle in the topmost (and if we have a choice, leftmost) point not covered by any of the rectangles we have already drawn.

An argument similar to the one used in the randomized solution proves that if we place a circle like that, it does not collide with any of the previously placed circles. As we place each point in the topmost available point, the centers of the previously placed circles are necessarily above the last one placed, and so if our new circle would collide with one of the previous ones, it would have to be in the rectangle we associated with it.

Now notice that the areas of all the rectangles we place is the sum of $8R^2 = 8 / \pi$ times the total area of the circles - which is easily less than the area of the mat. This means we always can place the next circle within the mat.

This solution is somewhat more tricky to code than the previous one (because we have to find the topmost free point, instead of just choosing a random one), but still easier than if we tried to actually stack circles (instead of replacing them by rectangles).

Analysis: Descending in the Dark

The main challenge in this problem is determining whether a cave is lucky. The set of all possible states is huge, so a complete search over all plans is simply not going to work. Fortunately, there is one observation we can use to greatly simplify the task.

Eliminating Backtracking

Fix a cave C , and recall that S_C denotes the set of squares from which C can be reached. We will build up our plan one instruction at a time.

Let X be the set of squares that you can be in if you start in S_C and follow the plan so far. If X contains a square not in S_C , we know the plan cannot work for every starting position. Otherwise, we can be 100% sure that the plan so far is fine! This is because the set of possible squares we are in has either stayed the same or gotten strictly smaller. In particular, if there was a plan that worked from the starting position, we can append it to what we have done so far, and it will still work.

So what does this mean? As long as we don't do a move that adds a square not in S_C to the set of possible squares X , we can go ahead and do that move, and it will still be possible to finish. In particular, we can always move left and right whenever we want, since moving left or right can never move you out of S_C .

All You Need Is Down!

Even with the previous observation, we still have work to do. We now know all the moves that can be done safely, but the state space is still huge. We can't find just any move; we need one that makes progress.

Here is where it is important that you cannot go up the mountain. Suppose you can add a Down move to the plan, satisfying the following two properties:

- There is at least one position in X from which you can actually move down. (Without this, the Down move will never do anything, and so is useless!)
- There is no position in X from which a down move will take you outside of X_C .

If you add this Down move to the plan, then the sum of the heights over all squares in X will have gone down, and it can never go up again, because you can never climb the mountain!

Since the sum of the heights is a positive integer, you will eventually have to stop making Down moves. At that point, you are stuck in one or more horizontal intervals. If there is just one interval and it contains the cave, the plan can be finished successfully. Otherwise, you're screwed! And remember, since this set of squares is a subset of what you started with, you were in fact screwed from the start.

Can You Go Down?

Only one question remains: given a set of positions X reachable from the start, can you come up with a valid plan that includes at least one Down move?

X must be contained in a set of horizontal intervals, bounded by impassable squares to the left and right. Since it is always safe to move left and right, we can keep moving left until X is actually just the leftmost square in each of these intervals. If we cannot make progress from that situation, we have already shown we are lost.

As we perform left and right moves from there, our position within each interval might change. However, note that if two intervals have the same length, our relative horizontal positions within them will always be the same. Therefore, let's define x_j to be our relative horizontal position within all intervals of length j . (In particular, $x_j = 0$ if we are in the leftmost position, and $x_j = j - 1$ if we are in the rightmost position.)

Lemma: It is possible to reach position $(x_1, x_2, \dots)$ using left and right moves if and only if $x_i \leq x_j \leq x_i + j - i$ for all $i < j$.

Proof: First we show $x_i \leq x_j$. This is true initially. If it ever failed after some sequence of moves, it would be because x_i and x_j were equal, and then either:

- We moved left, and only x_j was able to move.
- We moved right, and only x_i was able to move.

However, both of these scenarios are impossible. Therefore, x_i is indeed at most x_j , or in other words, the distance from x_i to the left wall is no larger than the distance from x_j to the left wall. The same argument can be applied for the right wall, which gives us the other half of the inequality: $x_j \leq x_i + j - i$.

Conversely, any set of positions with $x_i \leq x_j \leq x_i + j - i$ really can be reached via the following algorithm:

- Start with each $x_i = 0$.

- Try it yourself and you will see why it works!

```
old_safety = [SAFE] * (n+1)
for length in {n, n-1, ..., 1}:
    for i in [0, length-1]:
        pos_safety[i] = best(old_safety[i], old_safety[i+1])
        if moving down leaves SC:
            pos_safety[i] = UNSAFE
        elif moving down is legal and pos_safety[i] == SAFE:
            pos_safety[i] = SAFE_WITH_PROGRESS
    old_safety = pos_safety
return (pos_safety[0] == SAFE_WITH_PROGRESS)
```

Putting it all together, we repeatedly use the above algorithm to see if it is possible to make a down move. If so, we do it and repeat. Otherwise, we stop. If the only remaining interval is the one containing the cave, then that cave is lucky. Otherwise, it is not!

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Dark {
    static class Segment {
        int len;
        long goodExitMask;
        long badExitMask;
    }

    public static void main(String[] args) throws IOException {
        BufferedReader reader = new BufferedReader(new InputStreamReader(System.in));
        int numTests = Integer.parseInt(reader.readLine());
        for (int testId = 0; testId < numTests; ++testId) {
            String[] parts = reader.readLine().split(" ", -1);
            if (parts.length != 2) throw new RuntimeException();
            int rows = Integer.parseInt(parts[0]);
            int cols = Integer.parseInt(parts[1]);
            String[] field = new String[rows];
            for (int r = 0; r < rows; ++r) {
                field[r] = reader.readLine();
                if (field[r].length() != cols) throw new RuntimeException();
            }
            System.out.println("Case #" + (testId + 1) + ":");
            for (char caveId = '0'; caveId <= '9'; ++caveId) {
                int cr = -1;
                int cc = -1;
                for (int r = 0; r < rows; ++r)
                    for (int c = 0; c < cols; ++c)
                        if (field[r].charAt(c) == caveId) {
                            cr = r;
                            cc = c;
                        }
                if (cr < 0) continue;
                boolean[][] reach = new boolean[rows][cols];
                reach[cr][cc] = true;
                int nc = 1;
                while (true) {
                    boolean updated = false;
                    for (int r = 0; r < rows; ++r)
                        for (int c = 0; c < cols; ++c)
                            if (reach[r][c] && !reach[r+1][c] && field[r+1][c] == caveId) {
                                reach[r+1][c] = true;
                                updated = true;
                            }
                            if (reach[r][c] && !reach[r-1][c] && field[r-1][c] == caveId) {
                                reach[r-1][c] = true;
                                updated = true;
                            }
                            if (reach[r][c] && !reach[r][c+1] && field[r][c+1] == caveId) {
                                reach[r][c+1] = true;
                                updated = true;
                            }
                            if (reach[r][c] && !reach[r][c-1] && field[r][c-1] == caveId) {
                                reach[r][c-1] = true;
                                updated = true;
                            }
                }
                if (!updated) break;
            }
        }
    }
}
```

```

        if (reach[r][c]) {
            if (r > 0 && field[r - 1].charAt(c) != '#' && !reach[r - 1][c]) {
                reach[r - 1][c] = true;
                ++nc;
                updated = true;
            }
            if (c > 0 && field[r].charAt(c - 1) != '#' && !reach[r][c - 1]) {
                reach[r][c - 1] = true;
                ++nc;
                updated = true;
            }
            if (c + 1 < cols && field[r].charAt(c + 1) != '#' && !reach[r][c + 1]) {
                reach[r][c + 1] = true;
                ++nc;
                updated = true;
            }
        }
        if (!updated) break;
    }
    List<Segment> segments = new ArrayList<Segment>();
    for (int r = 0; r <= cr; ++r)
        for (int c = 0; c < cols; ++c)
            if (reach[r][c] && (c == 0 || !reach[r][c - 1])) {
                int c1 = c;
                while (reach[r][c1 + 1]) ++c1;
                Segment s = new Segment();
                s.len = c1 - c + 1;
                for (int pos = c; pos <= c1; ++pos) {
                    if (r + 1 < rows && field[r + 1].charAt(pos) != '#') {
                        if (reach[r + 1][pos])
                            s.goodExitMask |= 1L << (pos - c);
                        else
                            s.badExitMask |= 1L << (pos - c);
                    }
                }
                segments.add(s);
            }
    while (true) {
        int maxLen = 0;
        for (Segment s : segments)
            maxLen = Math.max(maxLen, s.len);
        long[] badByLen = new long[maxLen + 1];
        for (Segment s : segments) {
            badByLen[s.len] |= s.badExitMask;
        }
        long[] possible = new long[maxLen + 1];
        possible[1] = 1;
        for (int len = 1; len <= maxLen; ++len) {
            possible[len] &= ~badByLen[len];
            if (len < maxLen) {
                possible[len + 1] = possible[len] | (possible[len] << 1);
            }
        }
        for (int len = maxLen; len > 1; --len) {
            possible[len - 1] &= possible[len] | (possible[len] >> 1);
        }
        List<Segment> remaining = new ArrayList<Segment>();
        for (Segment s : segments)
            if ((s.goodExitMask & possible[s.len]) == 0) {
                remaining.add(s);
            }
        if (remaining.size() == segments.size()) break;
        segments = remaining;
    }
    System.out.println(caveId + ": " + nc + " " + (segments.size() == 1 ? "Lucky" : "Unlucky"));
}
}
}
}

```

Analysis: Mountain View

This problem gives you a lot of freedom in how you approach it, as it is OK to output any sequence of peak heights matching the input data. What follows is one of the possible solutions, but others are possible.

First notice that if from peak 3 we perceive peak 7 as highest, then from peaks 4, 5 and 6 I can surely see no farther than 7, as the peaks have to lie below the line connecting peaks 3 and 7, and no peak after 7 lies above that line (or it would appear higher from peak 3). If this condition is not satisfied, we can safely return "impossible".

So, begin with peak 1, then go to the one appearing to be highest from it, then the apparent highest from it, and so on. Give all these peaks height of 10^9 . Note this sequence necessarily ends on peak **N** (as it is strictly increasing).

Now sequentially take the first peak **A** that we haven't assigned a height to yet. As it's the first non-assigned, the previous peak (**A**-1) has necessarily been assigned, and we can look at the peak that appears highest from **A**-1. Suppose it's **B**. We know that when we start a sequence of apparently highest peaks from **A**, it has to contain at **B** - it cannot skip **B**, because the whole sequence has to lie below the [(**A**-1), **B**] line), and it is strictly increasing. If the sequence jumps over **B**, we return "impossible".

Assume the line [(**A**-1), **B**] has slope **T**; our construction will guarantee that **T** is an integer (notice the slope of the first line is zero). We will want all the peaks on the sequence starting at **A** to lie on a line passing through peak **B** and with slope **T**+1; this determines the line uniquely. If we do it this way, we will not spoil visibilities built before (because the whole line is below the line connecting **A**-1 and **B**), and in the sequence for each peak the apparently highest peak will be the next peak in the sequence (as they are all in one line, no peaks with constructed heights lie between **A** and **B**, and all the peaks after **B** are invisible, because they lie below the line connecting (**A**-1) and **B**, and thus, even more so, below the line connecting **A** and **B** (which has larger slope).

We continue in this fashion. We can check that at any moment of our construction:

- If we assume the peaks that we have not constructed yet do not stand in the way (have, say, a height of zero), then for each peak we already constructed the peak appearing to be the highest is the one we expect to appear highest.
- For any constructed peak **A**, either **A**+1 is constructed as well, or no peak between **A** and the peak visible from **A** has its height constructed.
- Adding a new sequence upholds these invariants, which proves that when we end, the visibilities are all OK.

Thus, the construction works.

Notice that we increase the slope by one for each sequence, so it is at most **N** at the end, and so the lowest we can get is $10^9 - N^2$; thus the heights are all positive.

Analysis: Swinging Wild

Translating to a graph problem

To analyse the problem, let us begin with describing the state we can be in at any particular point during traversing the swamp. It is easy to see that we need two items of information to describe this state - which vine are we currently holding on to, and how far away from the root are we holding it.

Once we have such a description, we can frame the problem as a path-finding problem in a graph. We start in the state $(0, \mathbf{d}_0)$, and we want to any state $(i, \mathbf{p})$ with $\mathbf{p} + \mathbf{d}_i \geq \mathbf{D}$ (to simplify, we can also add an artificial $\mathbf{N}+1$ st vine where our true love stands and demand that we reach the state $(\mathbf{N}, \mathbf{p})$ for any $\mathbf{p}$).

The transitions between states (or, in other words, edges in the graph) are allowed swings we can make. If we currently hold vine i , at $\mathbf{p}$ units away from the root, we can swing to any vine j rooted between $\mathbf{d}_i - \mathbf{p}$ and $\mathbf{d}_i + \mathbf{p}$, and the new length will be the minimum of $|\mathbf{d}_i - \mathbf{d}_j|$ and l_j . Note that the only use of climbing up a vine is to catch a vine that would be too short to be within our swing path - so we implicitly include it in the transitions described above (and thus do not need extra transitions from $(i, \mathbf{p})$ to $(i, \mathbf{p}-1)$).

Limiting the number of nodes

As described, we could have a large number of states (as there are very many possible vine lengths. A crucial fact to notice is that the second part of the state (the length away from root) is uniquely determined by the previous vine we got here from; and it is independent of where did we hold it (assuming we held it far enough to actually reach this state). This means that there are at most $\mathbf{N}^2$ states we will ever consider, as each is determined by a pair of vine indices.

We can now solve the small input. We have a graph with $\mathbf{N}^2$ nodes and at most $\mathbf{N}$ edges from each node, and we want to verify whether a path between some two nodes exists (we can merge all the target nodes into one node for simplicity). As we have at most $\mathbf{N}^3$ edges in total, any standard graph traversal algorithm (like BFS or DFS) will allow us to solve the problem.

Limiting the number of edges

For the large input, a $O(\mathbf{N}^3)$ solution will be unsatisfactory, and we need a bit more subtlety. There is a number of tricks one can use to beat down the complexity. One is to make use again of the fact that the target node depends only on the vine we start from, and not on the position at which we hold it. This means that there are in fact at most $\mathbf{N}$ edges from a given vine - if we manage to reach some vine j from a given vine i when holding it at position $\mathbf{A}$, we do not need to check this edge when considering moves from vine i held at position $\mathbf{B}$ (because even if we reach vine j , we will arrive in a state that we have already analysed). Thus, we need to make at most $\mathbf{N}^2$ edge traversals to visit all reachable nodes. There are various techniques to actually code this (for instance, for each vine, we could order the other vines by distance from this vine, and each time process this list from the closest vine and remove all traversed edges), we encourage you to explore the options.

An alternative for the large problem

An alternative is to notice another fact - the position part of the state (that is, the distance away from the root that you hold the vine at) never increases. This is because if you swing from vine i to vine j , it means you were holding i at least $|d_i - d_j|$ away from the root, while this quantity is at the same time an upper bound on the position you will hold vine j at.

This means that we can use Dijkstra's algorithm to find, for each vine, the maximum position we can hold this vine at - we treat the decreasing vine position as increasing time, and in each step we analyse what lengths could we obtain for each vine by moving from the current vine, and then choose the vine with the largest length to analyse. This will give us a $O(N^2 \log N)$ solution, which should be fast enough.

Going even faster

A fun fact is that this problem can be solved faster than $O(N^2)$, although you didn't need to notice this to solve our data sets. The key fact here is that if you can pass the swamp at all, you can always do it without going backwards (that is, you always catch a vine that's in front of you, you never swing back). An easy way to use this observation is to modify the Dijkstra's algorithm mentioned above to process the vines from first to last, which will turn $O(N^2 \log N)$ into $O(N^2)$.

To go down to $O(N)$, we need one more trick. Notice that if we can reach a vine, we will get the largest (meaning best) position if we swing to it from a vine that is as far away as possible. As we move only forward, this means that as soon as we can reach any particular vine, we should note the position achieved and we never need to check any other way of reaching it. This means we can get an $O(N)$ solution by keeping track of the vine we are currently processing and farthest we have reached so far, and from each vine trying to update only vines that we have not reached as yet. As each vine will be updated at most once, and read at most once, we will do $O(N)$ operations.

We encourage you to flesh out the details and try to prove the "never go back" lemma - it's not trivial!